

Recurrent Neural Networks

Encoder–Decoder: Sequence to Sequence

Introduction to Deep Learning · Class 27

Outline

From Reading to Writing

The Writer: a Language Model

The Encoder–Decoder

Training

The Bottleneck, and Attention

Decoding

Putting It Together

From Reading to Writing

Where we are coming from

Class 26: the recurrent network

what it gave us

one cell, applied at every step

a reader for sequences of any length

a state h_t carried from step to step

a memory of everything read so far

W reads, U remembers, V answers

one answer per step, from the state

back-propagation through time

the loss at any step reaches every earlier weight

- Every answer that network could give was tied to its reading: a label **for each** token, one label **for the whole** sequence, or the **next** token after the ones read
- Today's task is different in kind: the answer is a **new sequence** — with its own length, its own order, and no token-by-token correspondence to the input
- The instinct carries over — the same cell, the same state — but the network will have to **read first and write afterwards**, which needs two of them

Class 26; Jurafsky & Martin (2026), §14.7; Goodfellow et al. (2016), §10.4.

The problem: one sequence into another

- The running example, from Jurafsky & Martin: *the green witch arrived* → *llegó la bruja verde*
- Four words in, four out — but look at what moved: *arrived* is last in English and *llegó* **first** in Spanish; *green witch* becomes *bruja verde*, adjective after noun. No word sits where its translation sits
- Lengths differ too (*I am hungry*: three words; *tengo hambre*: two). Translation, summarising, answering a question, speech to text: all **a sequence in, a different sequence out**

The output cannot be produced token by token as the input is read: the first output word may depend on the last input word.

Jurafsky & Martin (2026), §14.7, §13.1; Goodfellow et al. (2016), §10.4.

Why the tools we have fall short

Class 26's reader as ...	what it produces	why it cannot translate
a tagger	one output per input, same position	<i>llegó</i> would have to come out at <i>the</i>
a classifier	one output for the whole input	a translation is not one label
a language model	the next word, from the words before	no source to translate; it only continues
a window	an output from a few nearby inputs	cannot see the last word when writing the first

- The tagger is closest and fails on exactly the point of the example: its outputs are **aligned** with its inputs
- The language model is the right kind of machine — it **writes** any length, one word at a time — but from nothing but its own previous words
- So the missing piece is not a new cell: it is a way to hand the language model **the whole source** before it writes a word

Jurafsky & Martin (2026), §14.3, §14.7; Class 26.

The idea: read everything, then write

- How a person translates: read the sentence **to the end**, form an understanding, then write in the target language's own order. Nobody writes *llegó* before reading *arrived*
- The equipment: **two** recurrent networks of Class 26. The first **reads** the source into its state — the **encoder**; its last state is the “understanding”, one vector, the **context**
- The second is a language model that **writes**, primed with the context instead of nothing — the **decoder**: one word at a time, each fed back, until an end symbol
- Nothing inside either cell changes; only the wiring does. The class is that wiring, its training, what is wrong with one context vector, and how the decoder picks its words

An encoder–decoder is a reader whose final memory becomes the starting memory of a writer.

Jurafsky & Martin (2026), §14.7; Goodfellow et al. (2016), §10.4; Cho et al. (2014); Sutskever et al. (2014).

Notation for today

$\mathbf{x}_{1:n}, \mathbf{y}_{1:m}$ source and target sequences

$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t$ the embedding of token t

$\mathbf{h}_t^e, \mathbf{h}_t^d$ encoder and decoder states

$\mathbf{W}, \mathbf{U}, \mathbf{V}$ input, recurrent, output weights

$g(\cdot)$ the recurrent cell, a black box today

$\mathbf{c}, \mathbf{c}_i$ the context: fixed, or per step

α_{ij} attention on source j at target step i

$\langle s \rangle, \langle /s \rangle$ start and end symbols

- Jurafsky writes $\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$; Goodfellow writes the same with $\mathbf{U}$ and $\mathbf{W}$ **swapped** and time as a superscript, $\mathbf{h}^{(t)}$. This block follows Jurafsky's letters with time as a subscript
- The August 2026 draft numbers the RNN chapter 14, not 13; citations here follow that draft

Jurafsky & Martin (2026), Eqs. 14.1–14.3; Goodfellow et al. (2016), Eqs. 10.8–10.11.

The Writer: a Language Model

The recurrent language model, in three lines

Definition 1 (RNN language model)

At step t , with $\mathbf{x}_t$ the one-hot of the current word,

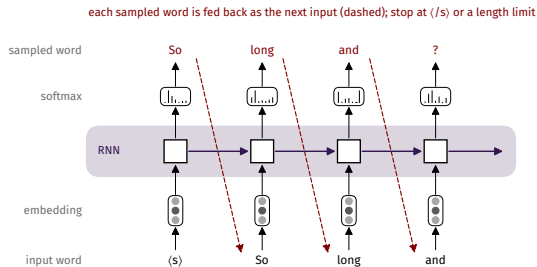
$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t, \quad \mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t), \quad \hat{\mathbf{y}}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t),$$

and $\hat{\mathbf{y}}_t[k] = P(w_{t+1} = k \mid w_1, \dots, w_t)$.

- Class 26's three jobs, with the answer fixed: **W** **reads** the word, **U** **remembers** the words before it, **V** **answers** "which word comes next?" with a distribution over the vocabulary
- On *The flights the airline was canceling*, the state still carries *flights*, so the answer can be *were*. The cell g stays a black box (Class 28); training on text is Class 29. Today the model is a **writer**

Jurafsky & Martin (2026), Eqs. 14.4–14.7, Fig. 14.5; Class 26; Mikolov et al. (2010).

Generating, one token at a time



Autoregressive generation: start from $\langle s \rangle$; the softmax gives a distribution over the next word; sample one (So); feed its embedding back as the next input; sample again (*long, and, ...*); stop at $\langle /s \rangle$ or a length limit. Every word is conditioned on the words the model itself chose, through the state. Every application today primes this loop with something richer than $\langle s \rangle$: the source. Layout of Jurafsky & Martin (2026), Fig. 14.9; §14.3.3; Shannon (1951).

The probability of a whole sequence

Theorem 2 (Chain rule of sequences)

For any distribution over sequences $\mathbf{y} = y_1, \dots, y_m$, conditioned on anything $\mathbf{x}$,

$$p(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^m p(y_t \mid y_1, \dots, y_{t-1}, \mathbf{x}).$$

An identity, not a modelling assumption.

- **Proof.** $p(y_1, y_2 \mid \mathbf{x}) = p(y_1 \mid \mathbf{x}) p(y_2 \mid y_1, \mathbf{x})$ by the definition of conditional probability; induct on m . □
- The recurrent model parameterises each factor by one softmax with the prefix carried in $\mathbf{h}_{t-1}$: its one assumption is that a fixed-size state can carry the prefix.
- So a generated sequence's probability is the product of the next-token probabilities it chose.

Jurafsky & Martin (2026), Eqs. 14.8–14.9, 14.28; Goodfellow (2016), Eq. 10.31.

The probability of a whole sequence

Theorem 2 (Chain rule of sequences)

For any distribution over sequences $\mathbf{y} = y_1, \dots, y_m$, conditioned on anything $\mathbf{x}$,

$$p(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^m p(y_t \mid y_1, \dots, y_{t-1}, \mathbf{x}).$$

An identity, not a modelling assumption.

- Proof. $p(y_1, y_2 \mid \mathbf{x}) = p(y_1 \mid \mathbf{x}) p(y_2 \mid y_1, \mathbf{x})$ by the definition of conditional probability; induct on m . □
- **The recurrent model parameterises each factor by one softmax with the prefix carried in $\mathbf{h}_{t-1}$: its one assumption is that a fixed-size state can carry the prefix.**
- So a generated sequence's probability is the product of the next-token probabilities it chose.

Jurafsky & Martin (2026), Eqs. 14.8–14.9, 14.28; Goodfellow (2016), Eq. 10.31.

The probability of a whole sequence

Theorem 2 (Chain rule of sequences)

For any distribution over sequences $\mathbf{y} = y_1, \dots, y_m$, conditioned on anything $\mathbf{x}$,

$$p(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^m p(y_t \mid y_1, \dots, y_{t-1}, \mathbf{x}).$$

An identity, not a modelling assumption.

- Proof. $p(y_1, y_2 \mid \mathbf{x}) = p(y_1 \mid \mathbf{x}) p(y_2 \mid y_1, \mathbf{x})$ by the definition of conditional probability; induct on m . □
- The recurrent model parameterises each factor by one softmax with the prefix carried in $\mathbf{h}_{t-1}$: its one assumption is that a fixed-size state can carry the prefix.
- **So a generated sequence's probability is the product of the next-token probabilities it chose.**

Jurafsky & Martin (2026), Eqs. 14.8–14.9, 14.28; Goodfellow (2016), Eq. 10.31.

The Encoder–Decoder

One change turns a language model into a translator

- Take the writer of the last section and give it a longer text to continue: the source, a separator, then the target:

the green witch arrived $\langle s \rangle$ llegó la bruja verde $\langle /s \rangle$

- Run the language model over it. During *the green witch arrived* its predictions are ignored — it is only **reading**, filling its state. From $\langle s \rangle$ on, its next-word distribution is **writing**, and every word it writes is conditioned on the source through that state
- That is the whole idea, and it already translates. What follows is giving the two halves names, drawing it, training it, and then fixing what is wrong with it

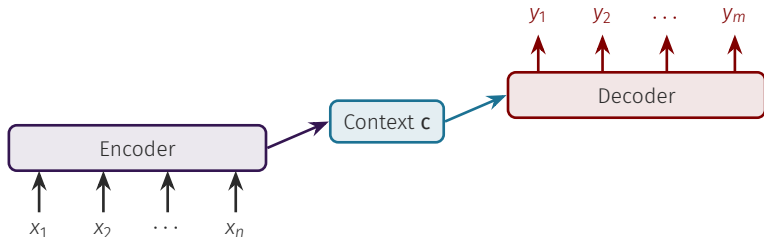
An encoder–decoder is a conditional language model: $p(\mathbf{y} \mid \mathbf{x})$ with the source read first.

Jurafsky & Martin (2026), §14.7, Eq. 14.31, Fig. 14.17.

The three parts

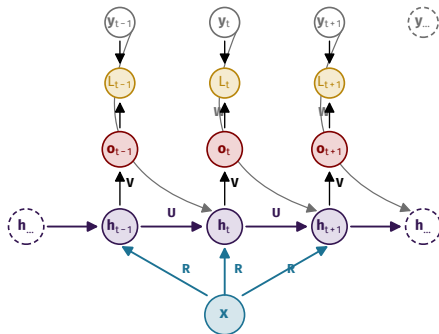
Definition 3 (Encoder-decoder network)

An *encoder* reads $x_{1:n}$ into states $h_{1:n}^e$. A *context* $c = f(h_{1:n}^e)$ conveys the source to the decoder. A *decoder* reads c and generates $h_{1:m}^d$, hence $y_{1:m}$, of any length. Either half may be any sequence architecture; here both are RNNs.



Jurafsky & Martin (2026), §14.7, Fig. 14.16; Goodfellow (2016), §10.4; Cho et al. (2014); Sutskever et al. (2014).

Two ways to hand over a vector: as an extra input

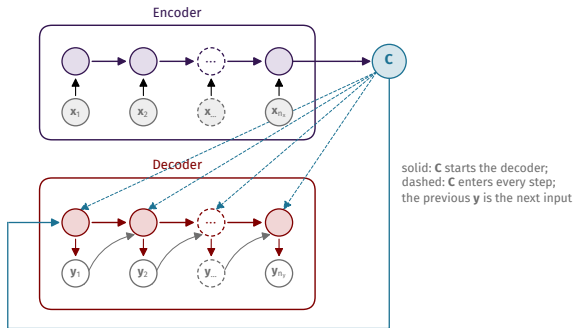


one vector x , the same at every step,
enters through R ;
the previous output y_{t-1} is the
sequence input, through W

Goodfellow's first way: a single fixed vector x — here, the context — is fed as an extra input to every step of the recurrence, through its own matrix R , while the sequence input at each step is the previous output y_{t-1} . The network maps one vector to a sequence, and the vector's influence is renewed at every step, so it does not fade as the output grows.

Layout of Goodfellow et al. (2016), Fig. 10.9; §10.2.4.

Two ways to hand over a vector: as the first state



Goodfellow's second way, and the encoder-decoder itself: an encoder RNN reads $x_1 \dots x_{n_x}$ and emits a context C , usually its final state; a decoder RNN starts from C as its initial state (solid) and generates $y_1 \dots y_{n_y}$, each output fed back as the next input. The dashed arrows are the first way applied here too: C can also enter every decoder step. Jurafsky's Eq. 14.32 uses both.

Layout of Goodfellow et al. (2016), Fig. 10.12; §10.4.

The equations

Proposition 4 (The basic RNN encoder–decoder)

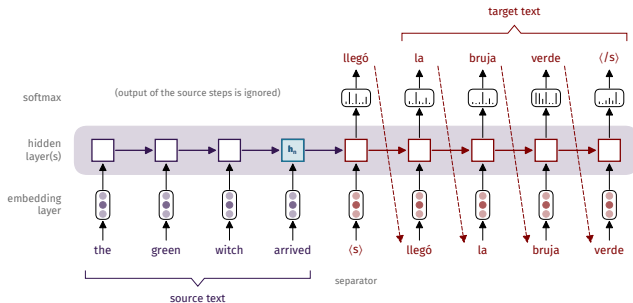
With g the recurrent cell and $\hat{\mathbf{y}}_{t-1}$ the embedding of the previous output,

$$\mathbf{c} = \mathbf{h}_n^e, \quad \mathbf{h}_0^d = \mathbf{c}, \quad \mathbf{h}_t^d = g(\hat{\mathbf{y}}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c}), \quad \hat{\mathbf{y}}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t^d).$$

- Two ways to hand over the context, and the equations use both: as the decoder's **initial state** $\mathbf{h}_0^d$, and as an **extra input at every step**, so its influence does not wane as the output grows
- The two previous drawings, combined: Goodfellow's extra input at every step (Fig. 10.9) and his first state (Fig. 10.12); Jurafsky's Eq. 14.32 is the sum of the two
- The encoder's own outputs are never used; in practice it is a stacked, bidirectional RNN (Class 26) and $\mathbf{h}_n^e$ is the top layer's last state

Jurafsky & Martin (2026), Eqs. 14.32–14.33, Fig. 14.18; Goodfellow et al. (2016), §10.2.4, Eq. 10.35.

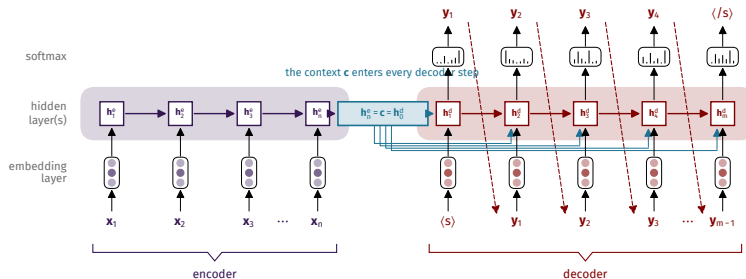
The whole model on the example



Translating the example at inference time: one recurrent network runs over source, separator and target. Each word is embedded and read into the hidden layer; the outputs of the four source steps are ignored; the last source state (teal) is what the target steps inherit. From $\langle s \rangle$ on, each softmax produces the next word, which is the next input (dashed), until $\langle /s \rangle$.

Layout of Jurafsky & Martin (2026), Fig. 14.17.

The encoder-decoder, unrolled



The same model with the two halves named, in the layout of Fig. 14.18. The encoder reads $x_{1:n}$ into $h_{1:n}^e$; its final state is the context c and the decoder's first state h_0^d ; the context also enters every decoder step (teal). The decoder reads $\langle s \rangle$ and then its own previous output, and emits $y_1, \dots, y_m, \langle /s \rangle$ through a softmax at each step.

Layout of Jurafsky & Martin (2026), Fig. 14.18; Eqs. 14.32–14.33.

Training

Teacher forcing

Theorem 5 (Teacher forcing is maximum likelihood)

Feed the decoder the gold token y_{t-1} at every step, not its own output, and sum the cross-entropy of each gold token:

$$L(\mathbf{x}, \mathbf{y}) = - \sum_{t=1}^m \log \hat{y}_t[y_t] = - \log p(\mathbf{y} \mid \mathbf{x}).$$

Minimising it maximises the likelihood of the target.

- **Proof.** With gold inputs, step t 's softmax is the factor $p(y_t \mid y_{<t}, \mathbf{x})$ of Theorem 2; the sum of the logs is the log of the product. □
- The steps decouple, so all m losses come from one encoder pass; and training is end to end, the loss flowing through the decoder and $\mathbf{c}$ into the encoder.

Jurafsky & Martin (2026), §14.7.1, Fig. 14.19; Goodfellow et al. (2016), §10.2.1, Eqs. 10.15–10.16; Williams & Zipser (1989).

Teacher forcing

Theorem 5 (Teacher forcing is maximum likelihood)

Feed the decoder the gold token y_{t-1} at every step, not its own output, and sum the cross-entropy of each gold token:

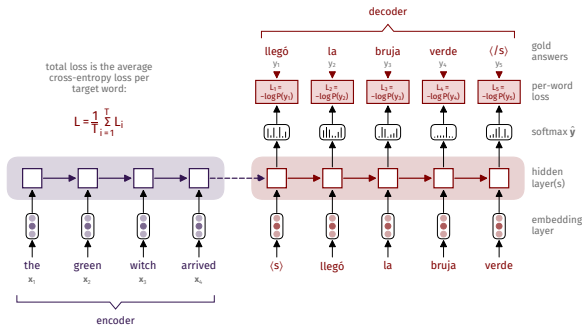
$$L(\mathbf{x}, \mathbf{y}) = - \sum_{t=1}^m \log \hat{y}_t[y_t] = - \log p(\mathbf{y} \mid \mathbf{x}).$$

Minimising it maximises the likelihood of the target.

- Proof. With gold inputs, step t 's softmax is the factor $p(y_t \mid y_{<t}, \mathbf{x})$ of Theorem 2; the sum of the logs is the log of the product. $\square$
- **The steps decouple, so all m losses come from one encoder pass; and training is end to end, the loss flowing through the decoder and c into the encoder.**

Jurafsky & Martin (2026), §14.7.1, Fig. 14.19; Goodfellow et al. (2016), §10.2.1, Eqs. 10.15–10.16; Williams & Zipser (1989).

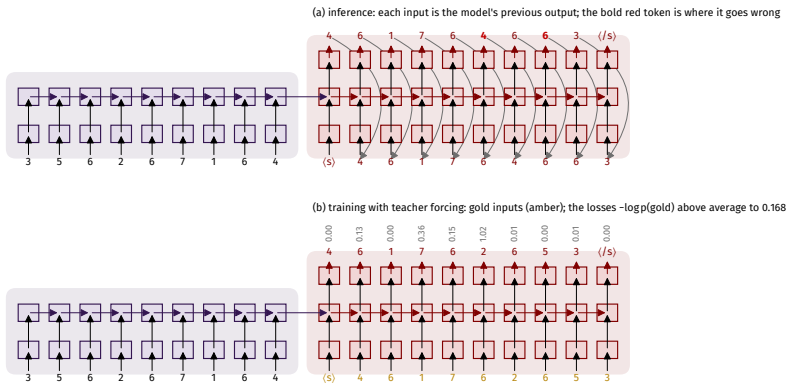
Teacher forcing, on the example



The encoder reads *the green witch arrived*; the decoder is fed the *gold* tokens $\langle s \rangle$ *llegó la bruja verde*, whatever it would have predicted; at every position the softmax is compared with the gold answer and pays $L_i = -\log P(y_i)$; the total loss is the average over the T target words, and its gradient flows back through the decoder and the context into the encoder.

Layout of Jurafsky & Martin (2026), Fig. 14.19; §14.7.1.

Training against inference, on the trained model



The same two regimes on a model trained here, on a task whose answers are checkable: a source of nine digits, to be written back reversed. (a) at inference the decoder's own outputs are fed back; at the sixth step it goes wrong and the rest follows. (b) in training the inputs are the gold tokens whatever the model said, and the per-token losses $-\log p(\text{gold})$, averaged, are the loss the gradient descends. Jurafsky & Martin (2026), Figs. 14.17 and 14.19.

What teacher forcing leaves out

- At inference the decoder uses its own estimated output $\hat{y}_t$ as the next input, so it tends to deviate more and more from the gold target as it keeps generating
- In training it was given the gold token instead, which speeds training but means the inputs it sees at test time differ from the ones it was trained on
- Goodfellow's remedy: mix free-running and teacher-forced inputs during training, so the network learns to handle its own errors
- The mismatch is visible two slides back: small losses on gold inputs, and a free-running output wrong from step six

Jurafsky & Martin (2026), §14.7.1; Goodfellow et al. (2016), §10.2.1.

Knowing when to stop

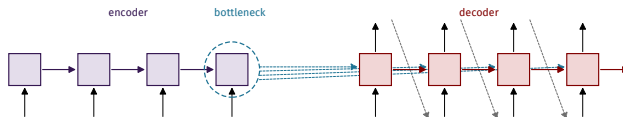
- A sequence model must also decide the **length** m . The usual way: a special symbol $\langle /s \rangle$ in the vocabulary, emitted when the sequence is done and never fed forward
- Equivalently, the model defines $P(\mathbf{y}_{1:m}) = P(m) P(\mathbf{y}_{1:m} \mid m)$ with $P(m)$ the probability of stopping at m ; Goodfellow's other options are a Bernoulli “stop” output at each step or predicting m up front
- The training data are parallel pairs — sentences aligned with their translations — with $\langle /s \rangle$ appended to every target, so the model learns to end where the data end

Goodfellow et al. (2016), §10.2.3, Eq. 10.34; Jurafsky & Martin (2026), §13.2.2.

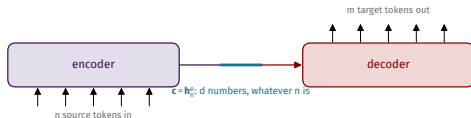
The Bottleneck, and Attention

Everything through one vector

(a) every arrow from the source into the decoder passes through one vector



(b) condensed: two blocks and one thin wire



(a) Every arrow from the source into the decoder passes through the encoder's last state, and every decoder step draws on that one vector. (b) Condensed: two blocks and one thin wire of d numbers, however many source tokens went in — a translator allowed one line of notes for any length of text, written last-word-first, since the state favours what it read most recently. Layout of Jurafsky & Martin (2026), Fig. 14.20; §14.8; Goodfellow et al. (2016), §10.4.

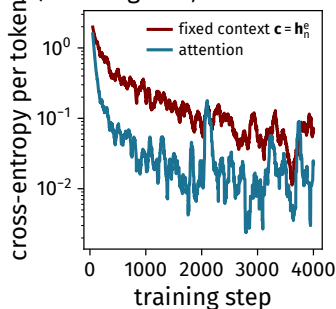
The bottleneck, measured: the experiment

- **What it means.** The decoder's only knowledge of the source is $\mathbf{c} = \mathbf{h}_n^e$: d numbers, the same for a four-word source and a forty-word one. If the source needs more than d numbers can hold, or the state has forgotten its beginning by the end (Class 26), the decoder cannot recover it
- **How to measure it.** Translation is hard to score; a task with one right answer is not. **Reverse a string of digits:** the first output needs the *last* input and the last output the *first*, so the whole source must survive in $\mathbf{c}$
- **The setup.** Two encoder-decoders of the same size ($d = 48$), trained here for 4,000 steps on lengths 2–10: one with the fixed context, one with attention (next slides); tested on new sources of length 2–16, scored by the fraction of outputs *exactly* right
- **What to expect.** If the bottleneck is real, the fixed model's accuracy should fall with n even inside its training range; a model that can look back at every encoder state should not

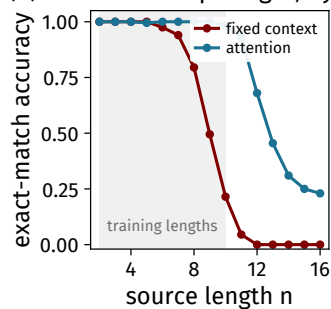
Jurafsky & Martin (2026), §14.8; Class 26.

The bottleneck, measured: the result

(a) training loss, same model size



(b) the whole output right, by length



(a) Training loss: both models learn, the attention model faster and lower. (b) Exact-match accuracy by source length, tested to 16. The fixed context (indigo) is perfect on short sources but at 80% by length 8 and 21% by length 10 — inside its training range: the vector is not carrying ten digits. Attention (maroon) is at 98% at 10 and still 68% at 12, beyond anything it trained on. The expected outcome, and a measurement of exactly what the next definition removes.

Jurafsky & Martin (2026), §14.8.

Attention, defined

Definition 6 (Dot-product attention)

At decoder step i , score every encoder state against $\mathbf{h}_{i-1}^d$, normalise, and average:

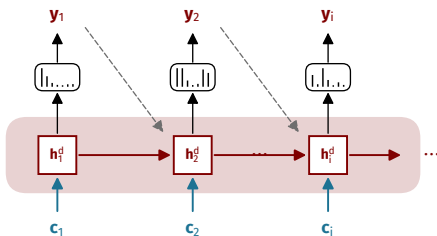
$$\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{i-1}^d \cdot \mathbf{h}_j^e, \quad \alpha_{ij} = \frac{\exp \text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e)}{\sum_k \exp \text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_k^e)}, \quad \mathbf{c}_i = \sum_{j=1}^n \alpha_{ij} \mathbf{h}_j^e,$$

and use $\mathbf{c}_i$ in place of $\mathbf{c}$: $\mathbf{h}_i^d = g(\hat{\mathbf{y}}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i)$.

- Still one vector, but a **different one at each step**, made from all the encoder states and chosen by the decoder
- The dot product needs equal dimensions; the **bilinear** score $\mathbf{h}_{i-1}^{d\top} \mathbf{W}_s \mathbf{h}_j^e$ learns what to compare and allows different ones

Jurafsky & Martin (2026), Eqs. 14.34–14.37, Figs. 14.21–14.22; Bahdanau et al. (2015); Luong et al. (2015).

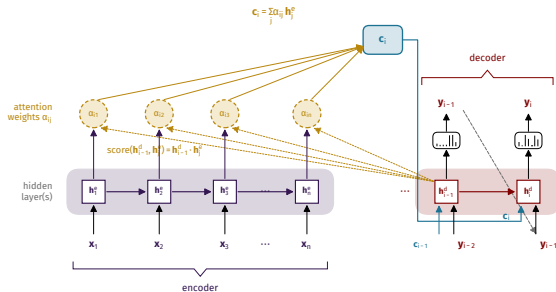
Attention, drawn: a context per step



a different context at every step:
 c_i is computed from all the
encoder states, for step i

The decoder of Fig. 14.18 with one change, in the layout of Jurafsky's Fig. 14.21: instead of one context c for every step, a different c_i enters each decoder step i . Writing *llegó* the decoder can look mainly at *arribado*; writing *bruja*, at *witch*. Where each c_i comes from is the next figure. Layout of Jurafsky & Martin (2026), Fig. 14.21.

Attention, drawn: one decoder step in full

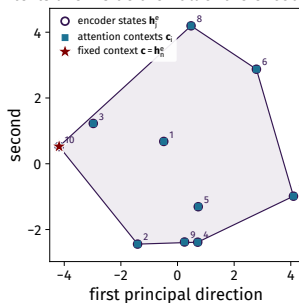


Computing c_i : the previous decoder state h_{i-1}^d is scored against every encoder state by a dot product (dotted); a softmax turns the scores into weights α_{ij} (amber) that sum to one; c_i is the encoder states averaged with those weights, and enters decoder step i (teal) beside h_{i-1}^d and the previous output. The weights are symbols here; on the reversal task the trained model puts nearly all of α_{ij} on $j = n + 1 - i$. Layout of Jurafsky & Martin (2026), Fig. 14.22; Eqs. 14.34–14.37.

Where the context can be

- The context is still a single vector, but now a weighted average of all the encoder hidden states, with weights that depend on the decoder's state
- It is generated anew at every decoding step, so it is different for each token produced
- On the reversal task the weights are nearly one-hot, so each c_i is, to a good approximation, one encoder state — a different one each step

contexts live inside the hull of the encoder states



Encoder states of one source, two principal directions; the contexts and the fixed context.

Jurafsky & Martin (2026), §14.8.

What attention is, and is not, today

- Attention here is a **fix to the bottleneck**: the decoder reads all the encoder states instead of the last, weighted by relevance to where it is
- It adds no recurrence and, with the dot product, no parameters; with $\mathbf{W}_s$, one matrix
- The weights α_{ij} are a by-product worth looking at: they say which source tokens produced which target token, an alignment nobody labelled
- Classes 30–31 will take this one idea, let every token attend to every other, and remove the recurrence altogether. Nothing in that needs more than Definition 6

A recurrent encoder–decoder with attention is the last recurrent architecture in the course, and the first attention.

Jurafsky & Martin (2026), §14.8; Bahdanau et al. (2015).

Decoding

Choosing the output

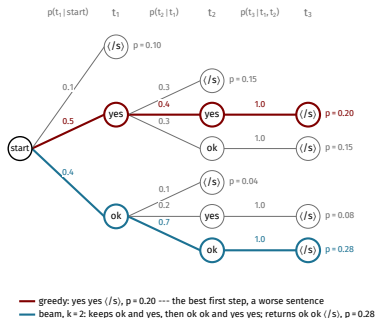
Definition 7 (Greedy and beam decoding)

The model gives $p(y_t \mid y_{<t}, \mathbf{x})$; decoding must pick one y . *Greedy* decoding takes $\hat{y}_t = \arg \max_w p(w \mid \hat{y}_{<t}, \mathbf{x})$ at each step. *Beam search* of width k keeps the k most probable prefixes at every step: extend each by every token, score the extensions by $\log p(\text{prefix}) + \log p(w \mid \text{prefix}, \mathbf{x})$, and keep the k best; a prefix that emits $\langle /s \rangle$ is complete.

- The most probable sequence is not the sequence of most probable tokens: a locally best token can lead only to bad continuations
- Exhaustive search over $|V|^m$ sequences is out of the question; greedy is $k = 1$; typical k is 5 to 10

Jurafsky & Martin (2026), §13.4, Fig. 13.7; Sutskever et al. (2014).

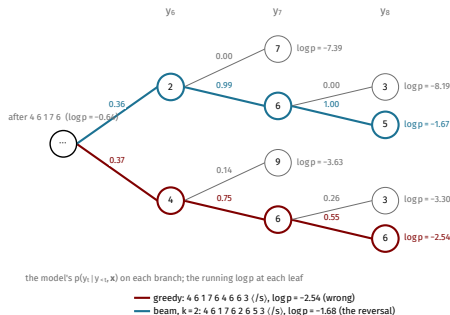
Greedy against beam, on a toy vocabulary



Jurafsky's search tree with the book's illustrative probabilities: a vocabulary of ok , yes , $\langle /s \rangle$, each branch labelled $p(t_k | t_{<k})$. Greedy (maroon) takes the best first word, yes at 0.5, and ends at probability 0.20. A beam of width 2 (teal) keeps ok and yes , extends both, keeps the two best prefixes, and returns $ok ok \langle /s \rangle$ at 0.28: a worse first step, a better sentence.

Layout and probabilities of Jurafsky & Martin (2026), Fig. 13.7; §13.4.

Beam search, run on the trained model



The same tree on the model trained here, source 3 5 6 2 6 7 1 6 4 to be reversed. Both decoders agree on 4 6 1 7 6; the tree starts there. At y_6 greedy takes 4 at 0.37 over 2 at 0.36 and cannot come back; a beam of width 2 keeps both, and the prefix through 2 then scores 0.99 and 1.00 where greedy's scores 0.75 and 0.55. Every number is the model's; the beam's output is the correct reversal, $\log p = -1.68$ against -2.54 . Jurafsky & Martin (2026), §13.4, Fig. 13.7.

What the beam costs, and what it does not buy

- At each step every prefix in the beam is extended by every token and only the k best extensions are kept, so the work grows with the beam width and the sequence length, not with the number of possible sequences
- A prefix that falls out of the top k is gone whatever its continuations would have scored, so the beam can still miss the most probable sequence; a wider beam misses less and costs more
- Longer sequences have more factors below one, so raw $\log p$ favours short outputs; the standard fix normalises by length, $\frac{1}{m} \sum_t \log p(y_t | \cdot)$
- Minimum Bayes risk decoding picks, from the candidates, the one most similar to the rest — one line, and beyond today

Jurafsky & Martin (2026), §13.4, Eq. 13.16, §13.4.1.

Putting It Together

One model, four settings

	context	decoder input	output choice
language model	$\langle s \rangle$ only	own previous output	sample or argmax
encoder-decoder	$\mathbf{c} = \mathbf{h}_n^e$, every step	own previous output	greedy or beam
... in training	the same	gold previous token	none: loss on gold
... with attention	$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j^e$	as above	as above

Every row is the same three equations of Definition 1 with a different context and a different source of the previous token. The recurrent cell g has not been opened; Class 28 opens it.

Takeaways

1. A sequence-to-sequence problem has an output of its own length and order; Class 26's reader cannot write it, but a language model can — given the source first
2. A recurrent language model is three equations, and generates by feeding its own output back (Definition 1)
3. Any distribution over sequences factorises token by token; the model parameterises the factors (Theorem 2)
4. An encoder–decoder is a conditional language model: read the source into a context, then generate (Definition 3, Proposition 4)
5. Teacher forcing trains it by maximum likelihood, one softmax per gold token (Theorem 5); it never sees its own mistakes
6. A fixed context is a bottleneck, and accuracy falls with source length; attention gives every step its own context, a convex combination of the encoder states (Definition 6)
7. Greedy decoding is not the most probable sequence; beam search keeps k prefixes and comes closer (Definition 7)

Class 28 opens the cell g: why a simple RNN forgets, and what an LSTM's gates change.

References i

1. D. Jurafsky & J. H. Martin, *Speech and Language Processing*, 3rd ed. draft, August 2026. Ch. 14 §14.2 (Eqs. 14.4–14.9, Fig. 14.5), §14.3.3 (Fig. 14.9), §14.7 (Eqs. 14.28–14.33, Figs. 14.16–14.19), §14.8 (Eqs. 14.34–14.37, Figs. 14.20–14.22); Ch. 13 §13.2, §13.4 (Fig. 13.7). The course plan’s “Chapter 13” is this chapter in the earlier draft.
2. I. Goodfellow, Y. Bengio & A. Courville, *Deep Learning*, MIT Press, 2016. §10.2.1 (Eqs. 10.15–10.16, Figs. 10.4, 10.6), §10.2.3 (Eq. 10.34), §10.2.4 (Eq. 10.35, Figs. 10.9–10.10), §10.4 (Fig. 10.12).
3. K. Cho et al., “Learning phrase representations using RNN encoder–decoder for statistical machine translation”, EMNLP 2014.
4. I. Sutskever, O. Vinyals & Q. V. Le, “Sequence to sequence learning with neural networks”, NeurIPS 2014.
5. D. Bahdanau, K. Cho & Y. Bengio, “Neural machine translation by jointly learning to align and translate”, ICLR 2015.
6. M.-T. Luong, H. Pham & C. D. Manning, “Effective approaches to attention-based neural machine translation”, EMNLP 2015.

References ii

7. R. J. Williams & D. Zipser, "A learning algorithm for continually running fully recurrent neural networks", *Neural Computation* 1(2), 1989.
8. S. Bengio, O. Vinyals, N. Jaitly & N. Shazeer, "Scheduled sampling for sequence prediction with recurrent neural networks", NeurIPS 2015.
9. M. Ranzato, S. Chopra, M. Auli & W. Zaremba, "Sequence level training with recurrent neural networks", ICLR 2016.
10. T. Mikolov et al., "Recurrent neural network based language model", Interspeech 2010.
11. C. E. Shannon, "Prediction and entropy of printed English", *Bell System Technical Journal* 30, 1951.

Questions?